Как писать самостоятельные APP на C и C++

Версия документа: 20.09.2026. Руководство для единого SDK ABI 6.

Здесь описан полный путь от `int main(void)` до файла `.APP`, который можно скопировать на MK61s и запустить. Приложение собирается отдельно от прошивки: её ELF, BIN и адреса функций для этого не нужны. Готовый APP работает на прошивках с совместимым API и загрузчиком ABI 6.

Содержание:

1 Модель приложения.

2 Первая программа и сборка.

3 Установка и запуск.

4 Сервисы прошивки.

5 Обработка файлов и графика.

6 Память, C++ и перенос существующего кода.

7 Сжатие, размеры и проверка.

Модель приложения

APP — нативная программа для Cortex-M4 Thumb с аппаратным соглашением вызова `hard-float`, `fpv4-sp-d16`. Её машинный код и данные загружаются в блок нужного размера у верхней границы свободной SRAM, ниже защиты стека. Размер одного APP вместе с BSS ограничен 20 480 байтами. Постоянного резерва SRAM такого размера в ABI 6 нет; блок выделяется при загрузке, с округлением размера до 32 байт. При нехватке свободной памяти загрузка завершается ошибкой. Адрес выбирает загрузчик; `0x20000000` служит виртуальной базой линковки. Таблица релокаций позволяет исправить внутренние указатели при загрузке. Одновременно исполняется один APP. Вызов синхронный: возврат из `main()` завершает приложение и возвращает управление прошивке.

Возможности устройства предоставляет таблица функций `mk61_api`. Например, вместо `printf()` программа вызывает `mk61_api->display_write_m8()`, а вместо платформенной клавиатуры использует `mk61_api->key_poll()`. SDK сам добавляет точку входа контейнера, подключает API и вызывает `main()`. Писать `mk61_module_entry`, startup или linker script не требуется.

Это freestanding C/C++: алгоритмы и небольшие библиотеки можно переносить, но программа для Windows/Linux с файловой системой, потоками и полным libc не заработает без адаптации. APP не является ELF-загрузчиком или средой ОС. Он содержит уже связанный образ, сжатый поток, компактную таблицу релокаций и 64-байтовый заголовок.

Штатные FOCAL, BASIC, WBMP, Markdown, CHIP-8, SETUP и USBDISK используют тот же ABI 6 и общий API. Дополнительные сервисы доступны любой программе через `mk61_app_get_services()`, объявленный в `mk61_app.h`. Набор возможностей зависит от сборки прошивки. Прежние имена System API остаются алиасами в адаптерах совместимости.

Первая программа и сборка

Инструменты

Нужны исходники этого репозитория, Python 3.10+ и ARM GCC. На macOS/Linux также нужен нативный C++17-компилятор для упаковщика; на Windows применяется встроенная Python-реализация. Проверен xPack ARM GCC 14.2.1 из STM32 Core. Для отдельного APP не нужны сборка Arduino-прошивки, CMake и Ninja. Это требование относится только к описанной здесь сборке произвольного пользовательского APP. Сборка прошивки и штатных System APP через Arduino IDE на Windows использует PowerShell/.NET и Python не запускает.

Добавьте каталог `bin` ARM GCC в `PATH`. Если это неудобно, передавайте сборщику --arm-toolchain-bin /путь/к/arm-gcc/bin. ARM-компилятор создаёт код для калькулятора; на macOS/Linux нативный компилятор собирает упаковщик, а на Windows упаковку выполняет Python. Можно явно указать готовый --packer /путь/к/mk61_module_pack.

Все дальнейшие команды выполняются из корня репозитория.

Исходник

Создайте каталог `my-app`, а в нём файл `main.c`:

```
#include "mk61_app.h"

int main(void) {
    const uint32_t required =
        MK61_APP_CAP_TEXT_DISPLAY | MK61_APP_CAP_KEYBOARD;
    if (!mk61_app_api_compatible(mk61_api, sizeof(*mk61_api), required))
        return MK61_APP_RUNTIME_ERROR;

    static const char message[] = "Hello, MK61s!";
    if (!mk61_api->display_clear() ||
        !mk61_api->display_write_m8(0, 0, message, sizeof(message) - 1))
        return MK61_APP_RUNTIME_ERROR;

    for (;;) {
        const int32_t key = mk61_api->key_wait();
        if (key == MK61_APP_KEY_OK || key == MK61_APP_KEY_ESC)
            return MK61_APP_OK;
    }
}
```

Программа показывает строку и ждёт `OK` или `ESC`. `key_wait()` обслуживает фоновые задачи во время ожидания, поэтому отдельный цикл задержки не нужен. Символ завершающего NUL не включается в длину текста.

Команда сборки

macOS/Linux:

```
python3 tools/apps/portable/build_portable_app.py --name HELLO \
  --source my-app/main.c \
  --output-dir .build/my-app
```

Windows, PowerShell или cmd, одной строкой:

```
python tools/apps/portable/build_portable_app.py --name HELLO --source my-app/main.c --output-dir .build/my-app
```

На Windows упаковщик запускается тем же Python, что и общий сборщик; отдельный нативный C++-компилятор не требуется. Имя после `--name` состоит из 1–31 ASCII-символа: первая буква или цифра, далее также допустимы `_` и `-`. Расширение `.APP` добавляется автоматически. Зарезервированные имена вроде `CON` и `NUL` не принимаются.

Результат в `.build/my-app`:

Файл	Назначение
`HELLO.APP`	Устанавливаемый сжатый контейнер.
`HELLO.elf`	Связанный ARM ELF для анализа символов и дизассемблирования.
`HELLO.map`	Карта размещения кода и данных.
`HELLO.bin`	Несжатый образ приложения без BSS. Это не прошивка прибора.
`HELLO.json`	Размеры, выбранное сжатие и версия компилятора.

На устройство нужен только `HELLO.APP`. Не переименовывайте ELF или BIN в APP и не прошивайте `HELLO.bin` через DFU.

Этот исходник проверен сборкой как C и как C++: с GCC 14.2.1 в обоих случаях получился APP 189 байт, образ вместе с BSS — 132 байта. Стек в эту цифру не входит.

Несколько исходников и C++

`.c` собирается как C11, `.cpp` — как C++17. Тот же пример можно сохранить в `main.cpp` и заменить путь в `--source`. Можно смешивать языки; каждый исходник перечисляется отдельным `--source`, каталог заголовков — отдельным `--include`. Если C++ вызывает собственные функции из C-файла, оформите общий заголовок с `extern "C"` под `#ifdef __cplusplus`.

Готовый смешанный проект — пример WBMP:

```
python3 tools/apps/portable/build_portable_app.py --name WBMP \
  --source examples/portable-apps/WBMP/main.c \
  --source examples/portable-apps/WBMP/viewer.cpp \
  --source code/wbmp.cpp --handled-magic I1 \
  --output-dir .build/my-wbmp
```

Для своего каталога заголовков добавьте `--include my-app/include`, для статической библиотеки — `--library my-app/lib/libalgo.a`. Библиотека должна быть собрана для того же Cortex-M4 Thumb, `hard-float`, `fpv4-sp-d16`, без зависимостей от отсутствующей ОС или libc. Объекты LTO требуют совместимого компилятора. Все необходимые реализации должны попасть в итоговую линковку: прямых импортов из ELF прошивки у обычного APP нет.

Установка и запуск

На устройстве должна быть актуальная прошивка ABI 6. Во всех штатных F401/F411 сборках загрузчик обязателен. Компоненты, выбранные в режиме APP, находятся в `/System`, встроенные не требуют файла, а ordinary APP запускаются без отдельного разрешающего флага. Linker script resident задаёт только границы свободной SRAM и защиты стека — массив под APP он не выделяет.

Установите программу через менеджер `tools/mkc.cmd` либо USB-диск:

1. На калькуляторе откройте Меню → USB-диск.
2. На диске `MK61S C8` создайте каталог `Apps` и скопируйте туда `HELLO.APP`.
3. Дождитесь окончания записи, безопасно извлеките диск на компьютере.
4. Нажмите `ESC` на калькуляторе, откройте Проводник и выберите APP клавишей `OK`.

Из USB-терминала эквивалентны команды:

```
open /Apps/HELLO.APP
run /Apps/HELLO.APP
```

Каталог `/Apps` — соглашение, а не требование загрузчика. Пользовательские APP можно перемещать и переименовывать; не занимайте канонические имена системных модулей в `/System`. При обновлении совместимой прошивки уже собранный ABI 6 не требует пересборки. Прошивка принимает только ABI 6: контейнеры ABI 2/3/4/5 нужно один раз пересобрать текущим SDK.

Сервисы прошивки

Публичные объявления находятся в `mk61_app.h` и `loadable_app_api.h`. Версия контейнера ABI 6, базовой таблицы API 1 и расширенных сервисов v2 — разные номера. SDK проверяет magic, версию и размер таблицы. Программа дополнительно проверяет нужные возможности через `mk61_app_api_compatible()`, как в примере. Не копируйте структуру API в собственный заголовок и не меняйте её порядок.

Capability MK61_APP_CAP_…	Функции и поведение
`TIME`	`millis_ms()`, `service()`, `delay_ms(ms)`.
`TEXT_DISPLAY`	`display_columns()`, `display_rows()`, `display_clear()`, `display_write_m8(col, row, text, bytes)`.
`KEYBOARD`	`key_poll()` возвращает событие либо `MK61_APP_KEY_NONE`; `key_wait()` ждёт нажатия.

Capability MK61_APP_CAP_…	Функции и поведение
KEY_STATE	key_pressed(key) проверяет удержание клавиши, в том числе несколько клавиш одновременно.
LED	led_set(enabled), led_blink(count, on_ms, off_ms).
SOUND	beep(hz, ms, volume_percent), sound_stop().
FILES	file_size(id), file_read(id, offset, output, length). Только чтение.
GRAPHICS	Проверка экрана, начало, передача и завершение графического кадра; подробнее ниже.

У текстовых координат начало (0, 0). Узнавайте количество строк и столбцов через API. Одна запись принимает до MK61_APP_MAX_TEXT_BYTES (63 байт) валидного M8 без NUL внутри строки. Каждый знак занимает ровно один байт. Строка выводится в пределах оставшихся столбцов, автоматического переноса на следующую строку нет. Для длинного текста разбивайте его на строки. Исходные UTF-8-литералы с не-ASCII текстом нужно преобразовать в M8 на этапе сборки либо записать M8-байтами явно.

display_clear(), display_write_m8(), led_blink(), beep() и операции начала/передачи графики возвращают 1 при успехе, 0 при отказе. Проверяйте результат: наличие capability не гарантирует успех операции. Это соглашение отличается от результата main() и файлового обработчика, где нулевой MK61_APP_OK означает успех.

Клавиши и длинные вычисления

Используйте MK61_APP_KEY_DIGIT_0…DIGIT_9, LEFT, RIGHT, OK, ESC и остальные имена из заголовка. Они обозначают логические клавиши, а их физическую раскладку выбирает прошивка. Для переносимого приложения не используйте аппаратные scan-code. Неизвестная клавиша кодируется как MK61_APP_KEY_RAW_BASE + scan_code.

key_poll() выдаёт события нажатия, а не отпускания. Для удержания нужен key_pressed(). Оба вызова обслуживают фоновые задачи; key_wait() делает это в своём цикле ожидания.

Во время длительных вычислений разбивайте работу на небольшие порции и регулярно вызывайте service(). Он обслуживает прошивку, но не завершает за вас приложение по ESC: обработку выхода нужно написать явно. Вместо пустых циклов задержки используйте delay_ms(). Для интервалов корректна беззнаковая разность (uint32_t)(mk61_api->millis_ms() - started_at): счётчик миллисекунд переполняется. Перед использованием времени проверьте MK61_APP_CAP_TIME.

Обработка файлов и графика

APP как просмотрщик файла

Обычный запуск вызывает main(). Чтобы приложение также открывало файлы, реализуйте необязательную функцию:

```
uint32_t mk61_app_open_file(uint32_t file_id);
```

В C++ определение можно явно оформить как extern "C" uint32_t mk61_app_open_file(uint32_t file_id). Декларация уже есть в mk61_app.h. Без собственной реализации SDK возвращает MK61_APP_INVALID_FILE.

При сборке задайте --handled-magic I1 для WBMP, T2 для Markdown либо другой поддерживаемый тип C6. Это двухсимвольная метка типа файла в C6, а не первые байты его содержимого. Параметр не регистрирует произвольное новое расширение: соответствие имени файла типу должно знать само C6.

При открытии файла прошивка передаёт обработчику file_id. Для чтения не нужны путь и fopen():

Вызов	Результат
file_size(file_id)	Размер в байтах; UINT32_MAX при ошибке. Ноль означает пустой файл.

Вызов	Результат
`file_read(file_id, offset, buffer, length)`	Число прочитанных байт; `UINT32_MAX` при ошибке. Для точного чтения сравнивайте результат с `length`.

Текущая реализация принимает ID, смещение и длину в диапазоне 0…65535. Файлы удобно читать порциями; проверяйте длину до разбора заголовка и не доверяйте размерам из содержимого. Создание, запись, удаление и поиск файлов доступны через дополнительные сервисы `MK61_SERVICE_CAP_FILES`, описанные ниже.

Обработчик возвращает один из кодов `MK61_APP_OK`, `MK61_APP_INVALID_FILE`, `MK61_APP_UNSUPPORTED_DISPLAY`, `MK61_APP_BUSY`, `MK61_APP_IO_ERROR`, `MK61_APP_RUNTIME_ERROR`. `main()` также возвращает ноль при успехе; ненулевой результат означает ошибку запуска, а не число для регистра X.

Встроенный обработчик и канонический System APP имеют приоритет перед пользовательским APP. Для проверки примера WBMP нужен комплект без штатных обработчиков WBMP/Markdown, иначе файл откроют они. Два пользовательских APP с одной сигнатурой создают неоднозначность. При прямом запуске примера `WBMP.APP` выводится подсказка; просмотр начинается при открытии самого `.wbmp`.

Графический экран

После проверки `MK61_APP_CAP_GRAPHICS` вызовите `graphics_available()`: графический код может быть собран, но текущий экран окажется символьным. Далее выполните `graphics_begin()` и проверьте его результат. Размер кадра узнаётся через `graphics_width()` и `graphics_height()`. Базовые функции `graphics_*` предоставляют кадр 192×64, то есть 1536 байт, для UC1609 или совместимого USB-экрана. Низкоуровневые сервисы дисплея имеют собственный контракт и проверку размеров экрана.

Байты расположены полосами высотой восемь пикселей, младший бит сверху:

```
/* frame обнулён, 0 <= x < width, 0 <= y < height. */
frame[(y / 8) * width + x] |= (uint8_t)(1U << (y % 8));
```

Передавайте весь кадр через `graphics_present(frame, frame_bytes)`. Между кадрами обслуживайте события. При изменении `graphics_revision()` завершите сеанс, заново проверьте экран и откройте графику ещё раз. После успешного `graphics_begin()` вызывайте `graphics_end()` на каждом пути выхода, включая ошибки. Полный пример такого цикла находится в `WBMP/viewer.cpp`.

Дополнительные сервисы для всех APP

Через общий API доступны запись файлов C6, рабочие буферы, редактор, меню, файловые диалоги, шрифты, регистры калькулятора, математика и настройки. Перед использованием запросите нужные возможности:

```
const mk61_app_services* services = mk61_app_get_services(mk61_api,
    MK61_SERVICE_CAP_MATH | MK61_SERVICE_CAP_FORMAT);
if (!services) return MK61_APP_RUNTIME_ERROR;
double root = services->math(MK61_SERVICE_SQRT, 144.0, 0.0);
```

`NULL` означает, что версия API или нужный сервис недоступны. Программа может показать сообщение, выбрать собственную реализацию или завершиться. Одни и те же сервисы доступны штатным и пользовательским APP. Например, в прошивке без языков сейчас отсутствуют общая математика и редактор; это определяется `MK61_SERVICE_CAP_*`, а не именем запускаемого APP.

Базовая таблица выросла со 104 до 108 байт: в конец добавлен `query_service`. Старые поля, номера и версия API 1 сохранены. `mk61_app_get_services()` проверяет размер до чтения нового поля. Новые APP требуют актуальную 108-байтовую таблицу; запуск на прежних прошивках не поддерживается. Для прямого обращения к новым полям проверяйте их наличие через `struct_size`.

Полный пример на C: SERVICES/main.c. Он вычисляет длину вектора и выводит результат через сервисы прошивки. `mk61_app_snprintf()` использует компактный форматировщик прошивки: целые числа, строки и символы. Вывод `double` через `%f`, `%e`, `%g` в текущих компактных сборках не включён.

```
python3 tools/apps/portable/build_portable_app.py --name SERVICES \
  --source examples/portable-apps/SERVICES/main.c --shared-runtime \
  --output-dir .build/portable-apps/services
```

Опция `--shared-runtime` заменяет включаемые в APP арифметические и строковые подпрограммы переходами в прошивку. При отсутствии совместимого сервиса startup завершится ошибкой до `main()`. Без этой опции нужные функции `libgcc` по-прежнему включаются в APP. Математика вызывается явно через `services->math`; `--shared-runtime` сам по себе не заменяет `sin()` или `sqrt()`.

Описание возможностей, параметров вызовов и пример записи файла находятся в справочнике сервисов.

Память, C++ и перенос существующего кода

Лимит одной APP составляет 20 КиБ: машинный код, константы, начальные значения `.data`, BSS и выравнивание. Сжатие уменьшает файл, но не это потребление RAM. Большой обнулённый массив почти не увеличивает APP на диске, однако целиком занимает BSS после загрузки.

Загрузчик арендует `memory_bytes`, округлённые до 32 байт, у верхней границы свободной RAM, непосредственно ниже защиты стека. Отдельного массива на 20 КиБ в `.bss` нет. Системная куча и временные буферы получают память снизу; загрузка и выгрузка APP сохраняют адреса и содержимое занятых буферов. Если свободного промежутка недостаточно, загрузка завершается ошибкой. После выгрузки весь блок снова доступен диспетчеру общей памяти.

Исполняющийся APP нельзя вытеснить или переместить. После возврата кэш System APP можно освободить при нехватке памяти. Адрес следующей загрузки может измениться. На F411 MPU разрешает исполнение только выделенного блока и отзывает разрешение при освобождении; стек остаётся защищённым.

Указатели на свои функции, строки, `.data` и BSS допустимы, включая поля упакованных структур. SDK получает адресные исправления из ELF, а не угадывает их по содержимому. Не записывайте числовые адреса своей SRAM в исходник и не сохраняйте указатели APP между загрузками. Внешние статические ARM-библиотеки собирайте с `-mword-relocations`; неподдерживаемый тип релокации останавливает сборку. ELF и таблица нужны сборщику, на устройство переносится только `.APP`.

Стек общий с прошивкой и находится за пределами выделенного блока APP. Не считайте всю свободную SRAM личным стеком приложения. Избегайте больших локальных массивов и глубокой рекурсии; буферы с известным сроком жизни можно сделать статическими, учитывая лимит BSS. При наличии `MK61_SERVICE_CAP_MEMORY` APP может арендовать WORKSPACE или SCRATCH под владельцем `MK61_SERVICE_OWNER_APP`. Это общие буферы: аренда может не удаться. Освобождайте их на всех путях выхода; перед файловой операцией, диалогом или другим сервисом освобождайте буфер, который тому может понадобиться. Указатель действителен только до освобождения аренды. Данные обычного APP в этих буферах не сохраняются между запусками.

Повторный запуск того же inode и заголовка использует общий вытесняемый кэш: `.data` и BSS сохраняются между командами одинаково у APPLICATION и System APP. Если образ вытеснен ради staging-буфера или другого APP, при следующем запуске `.data` восстанавливается, BSS обнуляется и снова вызывается INITIALIZE. Долговременное состояние всё равно храните в C6 или публичном workspace.

Поддерживаются структуры, классы и шаблоны, не требующие отсутствующего runtime. Допустимы константы и глобальные данные со статической инициализацией. Глобальные конструкторы/деструкторы запрещены линкером. Исключения, RTTI и поддержка потокобезопасной инициализации static отключены. Не используйте `std::string`, контейнеры с динамической памятью, `iostream`, `new`/`delete`, `malloc`/`free` без собственной подходящей реализации.

SDK добавляет `memcpy`, `memmove`, `memset`, `memcmp` и необходимые арифметические функции `libgcc` (либо общие подпрограммы с `--shared-runtime`). Полный libc и libm автоматически не подключаются: даже доступный при компиляции заголовок не означает, что реализация `printf`, `strlen`, `sin` или `fopen` войдёт в APP. При переносе небольшую необходимую функцию можно включить исходником или ARM-библиотекой, после чего проверить размер и отсутствие внешних зависимостей.

Практический порядок переноса:

1. Отделите алгоритм от ввода-вывода и платформенных функций.
2. Замените экран, клавиши, время и чтение файла вызовами `mk61_api`.
3. Уберите зависимость от ОС, динамических глобальных объектов и большого heap.
4. Добавьте `main()` и при необходимости `mk61_app_open_file()`.
5. Перечислите исходники в команде сборки, проверьте `memory_bytes` и ошибки линкера.

6 Проверьте повторный запуск, выход и обработку неподдерживаемого экрана.

При переносе прежнего пользовательского ABI 2 вместо собственного `mk61_module_entry` используйте startup SDK; уберите resident ELF, `--just-symbols` и manifest-сборку. Вызовы прежнего C++ API заменяются эквивалентными полями `mk61_api` и именами MK61_APP_… из C-заголовка. Адаптеры штатных модулей используют общий API, сохраняя прежние имена C++ функций только внутри исходников. Для нового C-приложения они не нужны.

Сжатие, размеры и проверка

Сборщик сам связывает ELF, извлекает образ и упаковывает APP. Для ABI 6 проверяются два варианта: ZX0 и ARM Thumb BCJ → ZX0. BCJ обратимо преобразует команды перехода перед сжатием. Выбирается меньший поток; при равенстве — обычный ZX0. Приложению не нужны код распаковщика, ручная настройка фильтра или дополнительный буфер распаковки. После сжатого потока записывается таблица релокаций; она входит в CRC контейнера. Загрузчик проверяет исходный образ до изменения адресов и не вызывает точку входа при любой ошибке таблицы.

В отчёте `NAME.json`:

Поле	Что измеряет
`image_bytes`	Несжатый образ кода и начальных данных.
`bss_bytes`	Обнуляемый хвост памяти, включая выравнивание.
`memory_bytes`	`image_bytes + bss_bytes`, максимум 20 480.
`app_bytes`	Заголовок 64 байта, сжатый поток и релокации; максимум 20 544.
`relocations` / `relocation_bytes`	Число адресных исправлений и размер их таблицы.
`compression`	`ZX0` или `ZX0+BCJ`; последнее означает BCJ перед ZX0.
`compiler`	Версия компилятора для воспроизведения сборки.

Размер APP-файла не включает метаданные C6. Стек, внутренние буферы прошивки и System workspace не входят в `memory_bytes`. Сравнение штатных модулей до и после перехода приведено в отчёте по ABI 4. Предыдущее сравнение ABI 2/3 сохранено в SIZE-COMPARISON.md.

Ошибки сборки и запуска

Сообщение или симптом	Что проверить
specify --arm-toolchain-bin…	ARM GCC отсутствует в PATH; укажите каталог его `bin`.
`undefined reference`	Не перечислен исходник/библиотека или вызвана функция вне SDK.
`unsupported global constructors/destructors`	Есть глобальный C++-объект с динамической инициализацией или деструктором.
`MK61 module exceeds the 20 KiB APP image limit`	Код + данные + BSS превысили максимум одного образа; это не означает фиксированный резерв SRAM.
`unsupported ARM relocation`	Пересоберите библиотеку с `-mword-relocations`; прямые ссылки за пределы APP запрещены.
`unpacked ELF section`	Исходник или библиотека создали секцию, не поддерживаемую linker script.
`PUT_APP` / `corrupt app`	Повреждён контейнер либо прошивка не поддерживает его формат; проверьте сборку и копирование.
`PUT_FIRMWARE` / `app/firmware mismatch`	Пересоберите старый ABI 2/3/4/5; для ABI 6 проверьте версию публичных сервисов.
Вместо своего просмотрщика открылся штатный	Сработал приоритет встроенного/System-обработчика.
ГРАФИКА / НЕДОСТУПНА	Активный дисплей не предоставляет нужный графический режим.
Кажется, что калькулятор завис	Проверьте вызовы `service()`, обработку выхода и расход стека.

Для проверки контейнера и SDK на компьютере:

```
MK61_TEST_SANITIZERS=1 bash tests/run_portable_app_tests.sh
```

Этот набор проверяет SDK и упаковку, а не произвольную новую программу. Свой алгоритм удобно отдельно проверять host-тестами с подменой таблицы API; пример для просмотра WBMP — `portable_wbmp_self_test.cpp`. На устройстве проверьте первый и повторный запуск, возврат через ESC, пустые и повреждённые входные файлы, смену экрана во время работы и восстановление обычного интерфейса после выхода.

Для воспроизводимого выпуска сохраните исходники, команду сборки, версию репозитория/SDK, версию toolchain и итоговые `.APP`/`.json`/`.map`. У ABI 6 не нужно прикладывать точный resident ELF для повторной сборки; требования к API и экрану приложения всё же следует указать.

Технический формат контейнера описан в `sdk/portable/README.md`, а все публичные расширенные сервисы — в APP-SERVICES.md. Manifest-сборка описана в руководстве по APP.